Particular	Details
Department	Institute of Computer Science and Engineering
Programme	B.E. / B.Tech.
Course Code & Course Name	CSA11 – Object Oriented Analysis and Design
Academic Year / Batch	2025
Faculty Name	Dr.Ramamoorthy M
Assignment Title	Object-Oriented Analysis and UML-Based Design of a Smart Warehouse Inventory and Order Fulfilment System (SWIFT)
Date of Issue	31.08.2026
Date of Submission	31.08.2026
Maximum Marks	100
Course Outcomes	CO1, CO2, CO3
Bloom's Taxonomy Level	BL3 – Apply, BL4 – Analyse, BL5 – Evaluate, BL6 – Create
SDG Mapping	SDG 9 – Industry, Innovation and Infrastructure; SDG 12 – Responsible Consumption and Production
Industry / Societal Relevance	Warehouse automation, logistics and supply-chain digitalisation, Industry 4.0, and software-based inventory/fulfilment management

Mandatory Application of Course Knowledge

Module / Concept	Application in the System
Module I — OOSDLC stages & methodology	SWIFT was carried through the OOSDLC stages of Requirement Analysis, OO Analysis, and OO Design (implementation is out of scope here). Agile–Scrum with Model-Driven Development (MDD) was selected over RUP because the system decomposes naturally into independent, incrementally deliverable modules — inventory, storage/bin allocation, order management, picking, and reporting — and because warehouse operators/pickers can give iterative feedback on each module as it is built. RUP's heavier, phase-gated ceremony was judged unnecessary for a system of this scope. Under MDD, the UML diagrams in this report (Class, Sequence, State Chart, Activity) are treated as the primary design artefacts that directly drive each sprint's implementation.
Module I — Encapsulation, Abstraction, Inheritance, Polymorphism	Encapsulation: Item.currentStock and StorageBin.currentOccupancy are private and can only be changed through validated methods (updateStock(), allocate()), preventing invalid states such as negative stock or over-allocated bins. Abstraction: callers invoke Order.confirmOrder() or PickingSession.completePicking() without needing to know how stock is checked or how the pick list is stored internally. Inheritance: StandardOrder and ExpressOrder both specialise the abstract Order class, inheriting its common fields (orderId, status) while each defining its own dispatch rule. Polymorphism: a single call such as order.getDispatchDeadline() returns a different SLA (24 hours vs 4 hours) depending on the concrete subtype the object actually is, without the calling code needing to check which type it has.
Module II — Use Case Diagram	Modelled in Section G.3.b (Figure 1): three actors (Customer, Operator, Picker) and nine use cases, with two include relationships (mandatory sub-steps — Check Stock Availability, Check Bin Capacity) and one extend relationship (the optional Report Exception path during picking).
Module II — Class Diagram	Modelled in Section G.3.b (Figure 2): association, composition (Order–OrderLineItem), generalisation (Order–StandardOrder/ExpressOrder) and multiplicities are all represented across the nine core classes.
Module II — Interaction Diagram	Modelled in Section G.3.b (Figures 3–4): two Sequence Diagrams — Place Order and Complete Picking Session — each showing lifelines, activation bars, and an alternate-path note.

Module / Concept	Application in the System
Module II — State Chart, Activity, Package, Component & Deployment	Modelled in Section G.3.b (Figures 5–8): the Order State Chart captures its full lifecycle; the Activity Diagram models the end-to-end fulfilment workflow with two decision points; the Package Diagram groups the system into five subsystems; the Component and Deployment Diagrams show the runtime/physical architecture (handheld scanner, application server, database, dashboard).
Module III — Noun/verb analysis	Candidate classes were identified by extracting the nouns in the problem statement (item, bin, order, line item, picking session, picker, customer, report) and candidate operations from the verbs (register, receive, allocate, place, pick, track, generate, confirm) — detailed in Section G.3.a.
Module III — Entity/boundary/control classification	Item, StorageBin, Order, OrderLineItem, PickingSession, Picker, and Customer were classified as entity objects; OrderUI and PickingUI as boundary objects; OrderController and PickingController as control objects — see the classification table in Section G.3.a.
Module III — Relationships, attributes, methods	Each class's attributes and methods were derived directly from the functional requirements in Section D1 (e.g. Item.reorderThreshold exists specifically because requirement D1.1 asks the system to flag reorder), and relationships/multiplicities were derived from the cardinality implied by those same requirements — detailed in Section G.3.a and the Class Diagram.

1. G.1 — Problem Understanding and Formulation

The Problem

The warehouse currently tracks inventory, storage bins, and orders manually using registers and spreadsheets. As order volume grows, this causes bin double-allocation, orders confirmed against out-of-stock items, missed dispatch SLAs, poor zone-utilisation visibility, uneven picker workload, and an inability to trace an order's status or analyse fulfilment performance. SWIFT is the proposed object-oriented software solution that manages inventory, storage allocation, order lifecycle, picking, and performance reporting to remove these manual-process failure points.

Stakeholders / Users

- **Operator (Warehouse/Inventory Clerk) —** registers items, receives stock, allocates storage bins, tracks order status, and generates fulfilment reports.
- **Picker —** executes picking sessions assigned to them and reports exceptions (damaged/missing items).
- **Customer —** places orders and tracks their own order status.

Information to be Stored

- Item: ID, name, category, unit price, reorder threshold, current stock.
- Storage Bin: ID, maximum capacity, current occupancy, stored item(s), availability status.
- Order: ID, customer reference, line items, priority, status, creation timestamp.
- Picking Session: pick-list ID, assigned picker, items to pick, status, start/completion time.
- Aggregated performance data for the fulfilment report (SLA compliance, zone utilisation, reorder list, average pick time).

Major Operations

- Register item, receive stock, allocate storage bin.
- Place order (with stock-availability check), confirm/cancel order, track order status.
- Create and complete a picking session, flag an exception.
- Generate a fulfilment performance report.

Assumptions

- A picking session is created against one Order as a whole, not against individual line items — a single picker completes all items on an order's pick list (Section G.6 discusses this simplification).
- The dispatch SLA values (24 hours standard / 4 hours express) are configurable parameters, not hard-coded constants, as permitted by the case study.
- A FulfilmentReport is treated as a derived/computed view over Order and PickingSession data rather than a persisted entity in its own right.

- Item-to-bin storage is simplified to one bin holding one or more units of a single item type at a time (no split-lot tracking across bins for this prototype).

Constraints to be Satisfied

- Functional: unique IDs for items/bins/orders/pick-lists; no bin over-allocation; no order confirmation exceeding available stock; a picker cannot hold two active pick-lists at once.
- Technical: standard UML 2.x notation; strictly object-oriented (not ER-centric); a recognised CASE tool (draw.io) used throughout.
- Performance: the design must scale to at least 200 items, 50 concurrent orders, and 10 storage zones without structural redesign.
- Sustainability: the design should support analysis of overstocking/wastage reduction and picker-travel optimisation (elaborated in Section G.7).

2. G.2 — Application of Course Knowledge

The Section E table above maps each of the nine required concepts to a concrete part of SWIFT; this section adds the reasoning behind two of the most consequential decisions.

Why Agile–Scrum with MDD, not RUP

RUP's four structured phases (Inception, Elaboration, Construction, Transition) with formal artefact gates suit large, contractually-specified systems where requirements are frozen early. SWIFT's five modules (inventory, bin allocation, order management, picking, reporting) are naturally independent enough to be built and demonstrated to warehouse staff one at a time, and operator/picker feedback after each module is likely to refine later modules — a fit for Agile–Scrum's short, feedback-driven sprints. MDD is layered on top so that the UML diagrams already produced (Class, Sequence, State Chart) are not just documentation but the actual blueprint each sprint implements against, keeping the design and the code from drifting apart as the system grows.

How OO Analysis Techniques Led to the Class Structure

Noun/verb analysis over the problem statement (Section G.3.a) surfaced the seven entity classes directly — every noun that persists data (Item, StorageBin, Order, OrderLineItem, PickingSession, Picker, Customer) became a candidate entity class, while every verb that represents a distinct responsibility (checkStock, allocate, confirmOrder, completePicking) became a candidate method attached to whichever entity owns the data that operation changes. Where a requirement explicitly implied a constraint — for example, 'must not confirm an order line item that exceeds available stock' — that constraint was pushed into a method (Item.checkReorderNeeded(), Order.confirmOrder()) rather than left as free-floating validation logic, keeping each rule encapsulated next to the data it protects. This is why the class list is not arbitrary: every class and every method traces back to a specific sentence in the case study, which is also what the Design Rationale in Section G.3.c argues.

3. G.3 — Solution Design and Implementation

G.3.a — Object-Oriented Analysis and Class Architecture

Noun/Verb Analysis

Analysis	Result
Nouns → Candidate Classes	item, storage bin/zone, order, order line item, picking session, picker, customer, (fulfilment) report
Verbs → Candidate Operations	register, receive (stock), allocate, place (order), confirm, cancel, pick, track, generate, flag (exception)

Entity / Boundary / Control Classification

Object	Classification	Justification
Item, StorageBin, Order, OrderLineItem, PickingSession, Picker, Customer	Entity	Each holds persistent data that outlives a single user interaction (stock levels, bin occupancy, order status).
OrderUI, PickingUI	Boundary	Represent the screen/interface a Customer or Picker interacts with directly; isolate presentation from business logic (used in the Sequence Diagrams).
OrderController, PickingController	Control	Coordinate the workflow between a boundary object and the entity objects it must update — e.g. OrderController mediates between OrderUI, Item, and Order during order placement.

Classes, Attributes, Operations, and Relationships

Class	Key Attributes	Key Operations
Customer	customerId, name	placeOrder()
Order (base)	orderId, priority, status, createdAt	confirmOrder(), cancelOrder()
StandardOrder / ExpressOrder	— (inherit Order)	getDispatchDeadline() — overridden: 24 hrs / 4 hrs
OrderLineItem	lineItemId, quantity	getSubtotal()
Item	itemId, name, unitPrice, reorderThreshold, currentStock	checkReorderNeeded(), updateStock()
StorageBin	binId, maxCapacity, currentOccupancy, status	checkAvailability(), allocate()
PickingSession	pickListId, status, startTime, completionTime	startPicking(), completePicking(), flagException()
Picker	pickerId, name, shift	getActiveSession()

Relationships and multiplicities (shown in Figure 2): Customer places Order (1 to 0..*); Order is composed of OrderLineItem (1 to 1..*, filled diamond, since a line item has no meaning outside its

order); OrderLineItem references Item (0..* to 1); Item is stored in StorageBin (1 to 0..*); Order is fulfilled by PickingSession (1 to 0..1); PickingSession is assigned to Picker (0..* to 1); StandardOrder and ExpressOrder generalise from Order (hollow-triangle arrows).

G.3.b — UML Diagram Set

All nine diagrams below were prepared with a UML/CASE tool (draw.io) and are internally consistent with each other and with the class list above — e.g. every message in the two Sequence Diagrams calls a method that actually exists on the class list, and the states in the State Chart are exactly the status values Order.status is allowed to take.

1. Use Case Diagram

Three actors — Customer, Operator, Picker — interact with nine use cases. Place Order includes Check Stock Availability (a mandatory sub-flow, since an order can never be confirmed without it), Allocate Storage Bin includes Check Bin Capacity for the same reason, and Report Exception extends Pick Order as an optional path only taken when an item is damaged or missing.

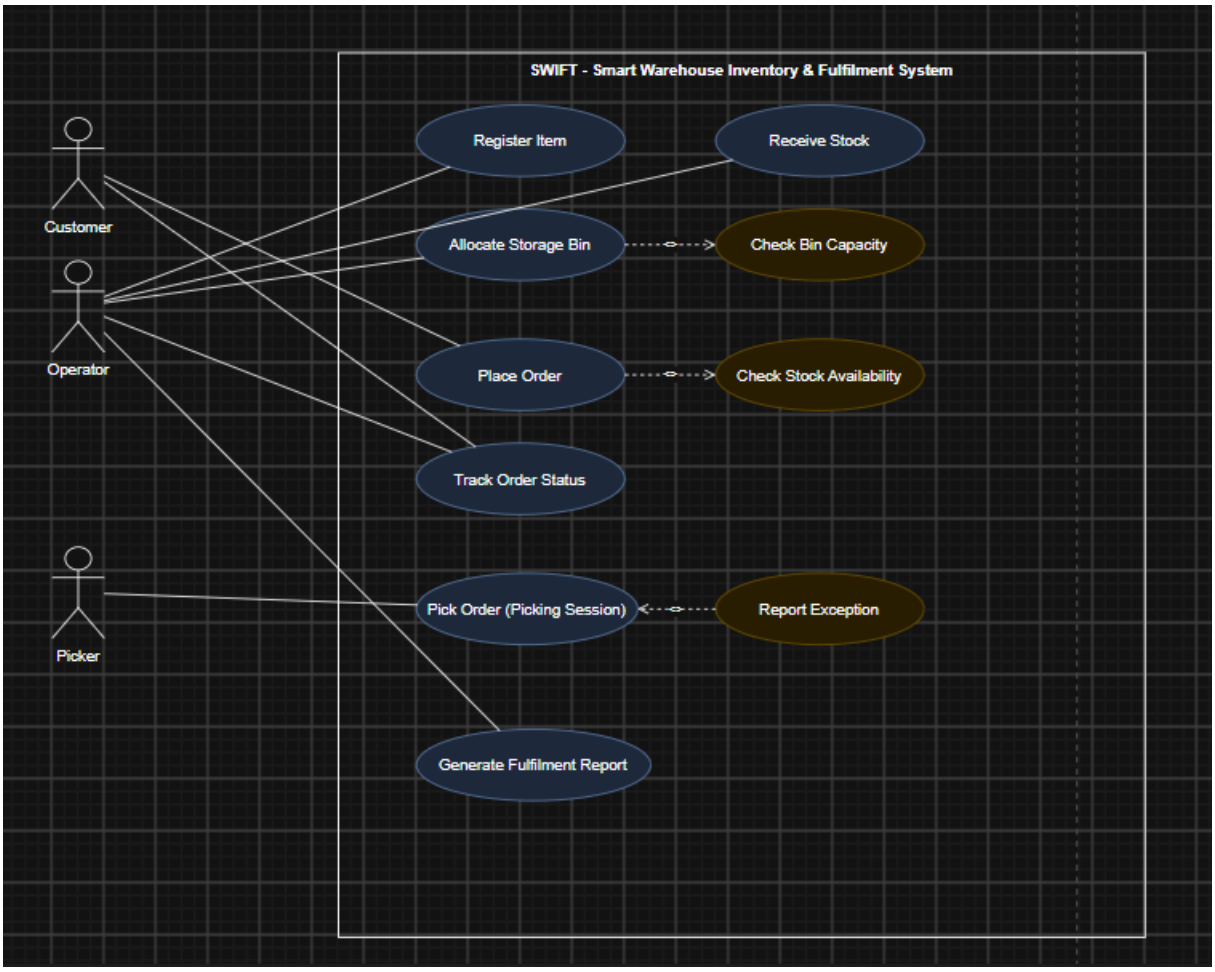

Figure 1 — Use Case Diagram (SWIFT)

2. Class Diagram

The nine classes and their relationships described in Section G.3.a, shown graphically with multiplicities, the Order/StandardOrder/ExpressOrder generalisation, and the Order–OrderLineItem composition.

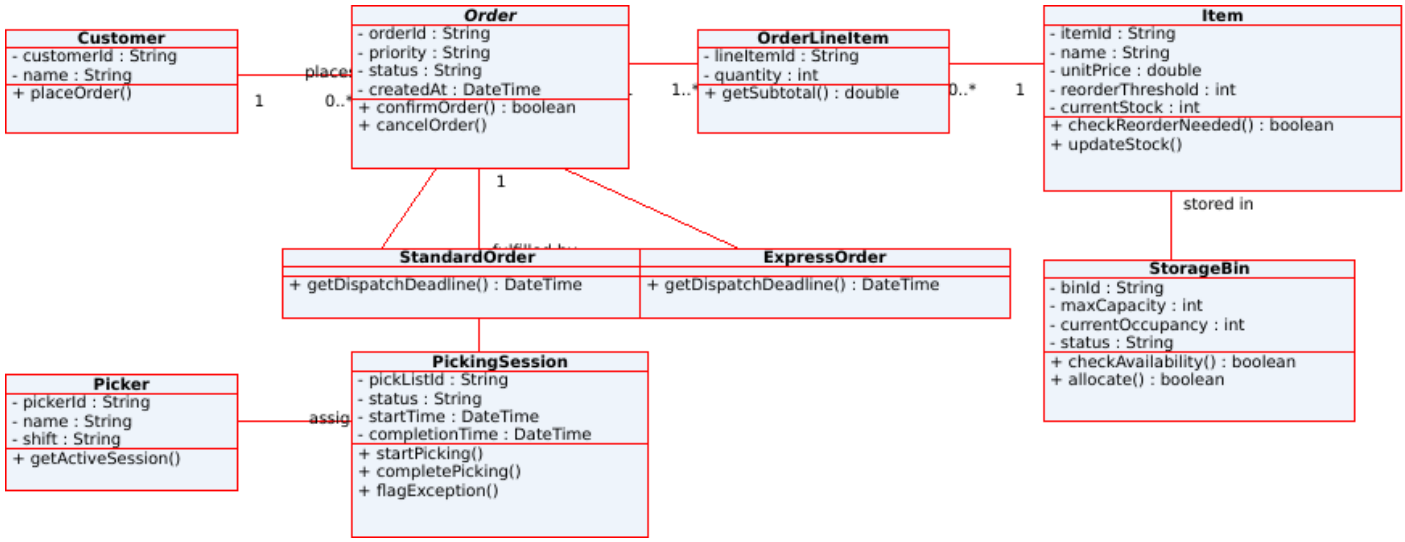

Figure 2 — Class Diagram (SWIFT)

3. Sequence Diagram — Place Order

Customer submits an order through OrderUI, which hands off to OrderController. The controller checks stock on Item for each line item before creating the Order; the boxed note documents the alternate path taken when stock is insufficient.

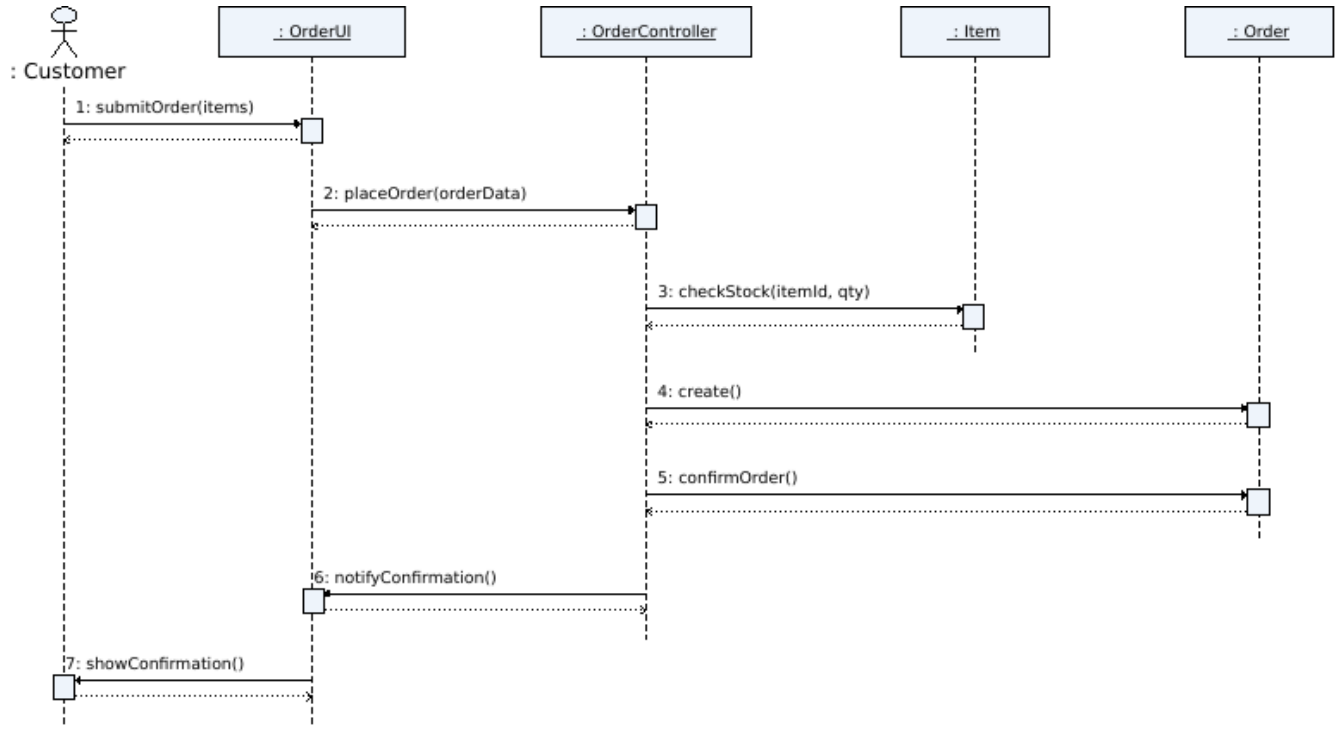

Figure 3 — Sequence Diagram: Place Order

4. Sequence Diagram — Complete Picking Session

The Picker marks items as picked through PickingUI; once all items are picked, PickingController marks the session complete and updates the Order's status to 'Packed'. The boxed note documents the exception path.

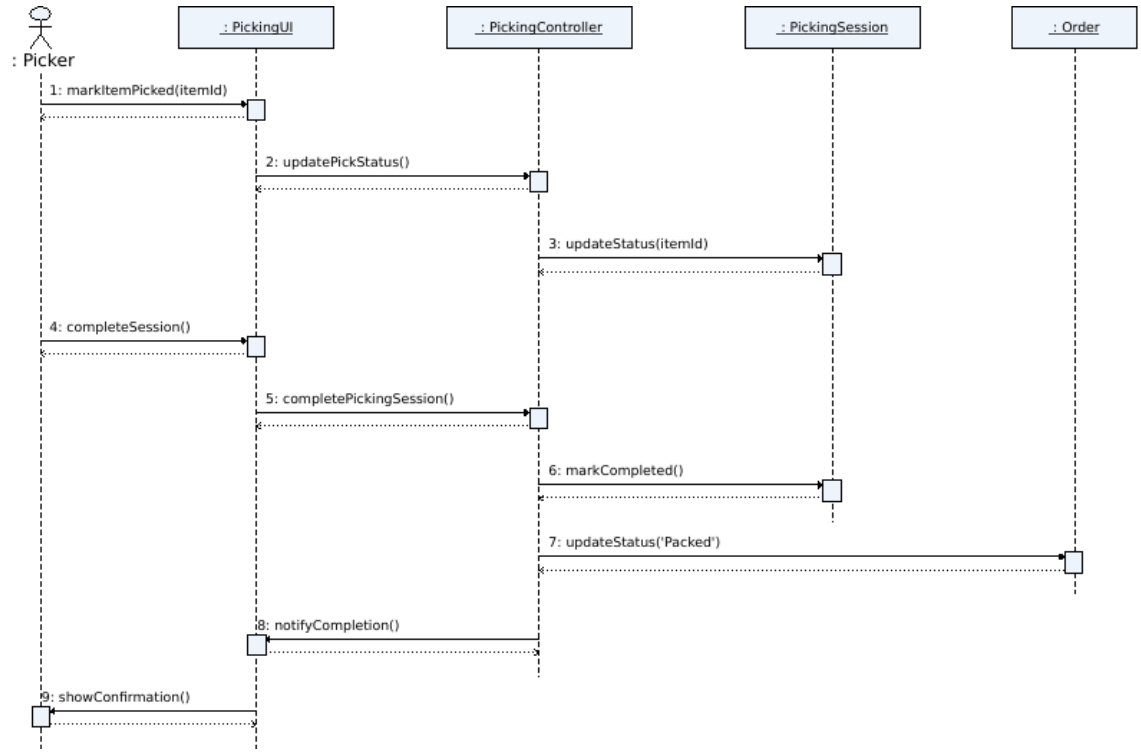

Figure 4 — Sequence Diagram: Complete Picking Session

5. State Chart Diagram — Order Lifecycle

The Order object moves through Placed → Allocated → Picking → Packed → Dispatched → Delivered on the happy path, with alternate transitions to Cancelled (from Placed or Allocated) and Exception (from Picking, with a retry path back to Picking or an escape to Cancelled).

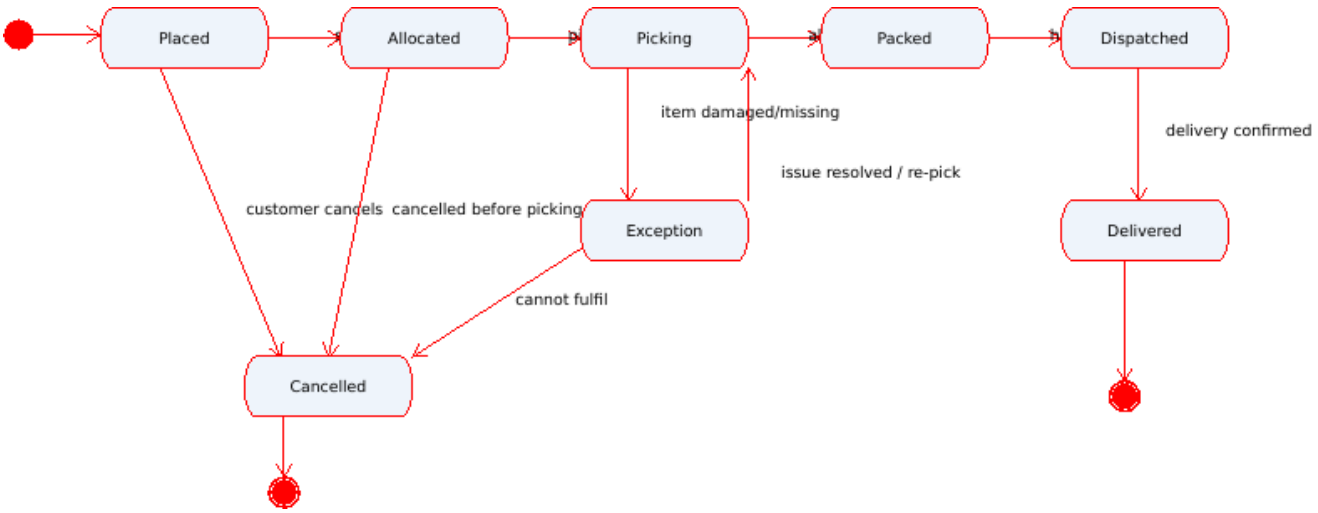

Figure 5 — State Chart Diagram: Order Lifecycle

6. Activity Diagram — End-to-End Order Fulfilment

Models the full workflow from Place Order to Deliver Order, with two decision points as required: a stock-availability check immediately after order placement, and an item-condition check during picking that can loop back for a re-pick.

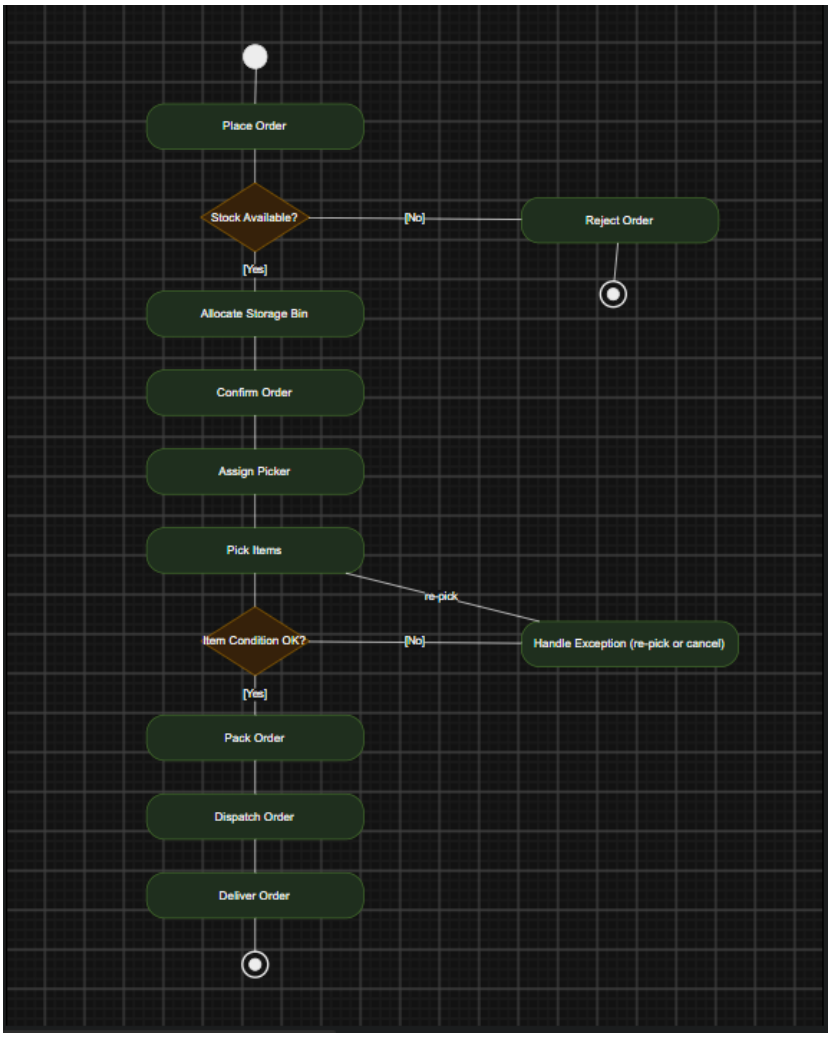

Figure 6 — Activity Diagram: End-to-End Order Fulfilment

7. Package Diagram

The system is grouped into five logical subsystems — Presentation, Order Management, Inventory Management, Picking & Fulfilment, and Reporting — with dependency arrows showing that Order Management depends on Inventory Management (to check stock), Picking & Fulfilment depends on both, and Reporting depends on both Order Management and Picking & Fulfilment for its source data.

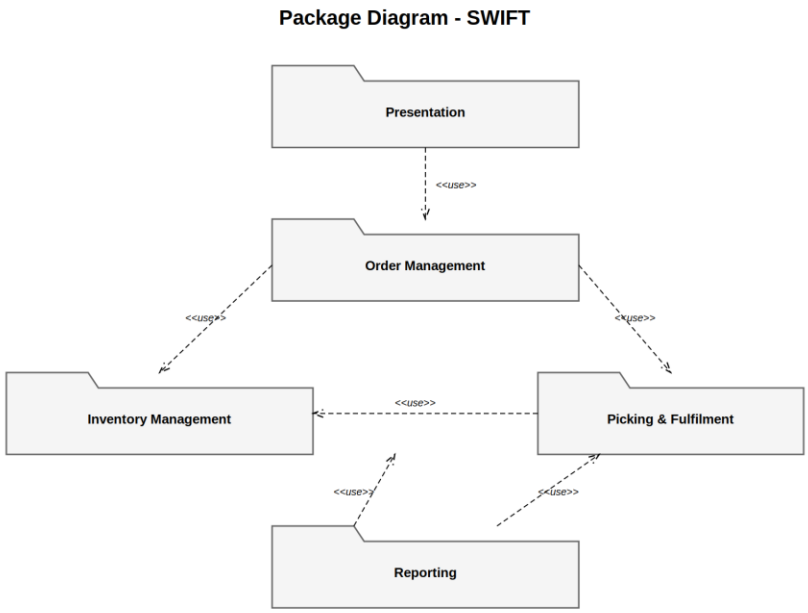

Figure 7 — Package Diagram

8. Component Diagram

The Warehouse Application Server exposes three internal services (Order, Inventory, Picking) as components with defined interfaces. The Handheld Scanner App and Reporting Dashboard are external components that depend on these interfaces rather than on the server's internal implementation, and all three services read/write to the shared OODBMS.

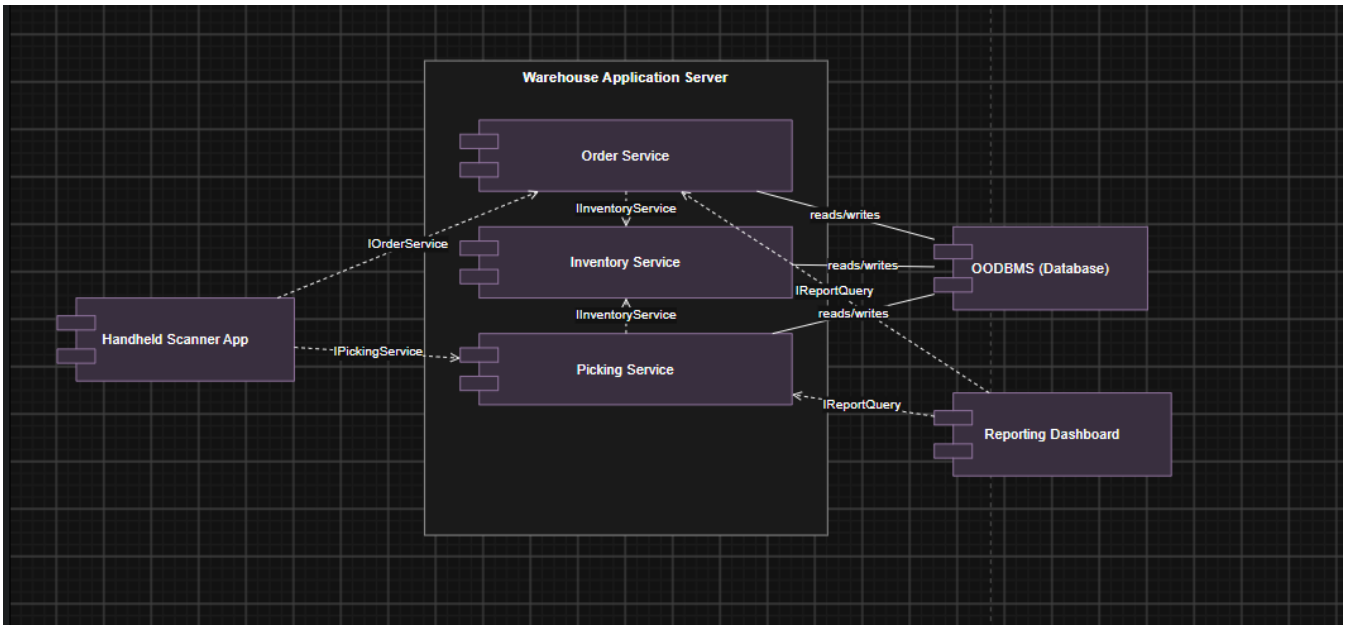

Figure 8 — Component Diagram

9. Deployment Diagram

Shows the physical/runtime nodes: a Handheld Scanner device communicating over HTTPS/REST with the Warehouse Application Server, which in turn talks to the Database Server over JDBC/ODBC, and a browser-based Reporting Dashboard also communicating over HTTPS/REST.

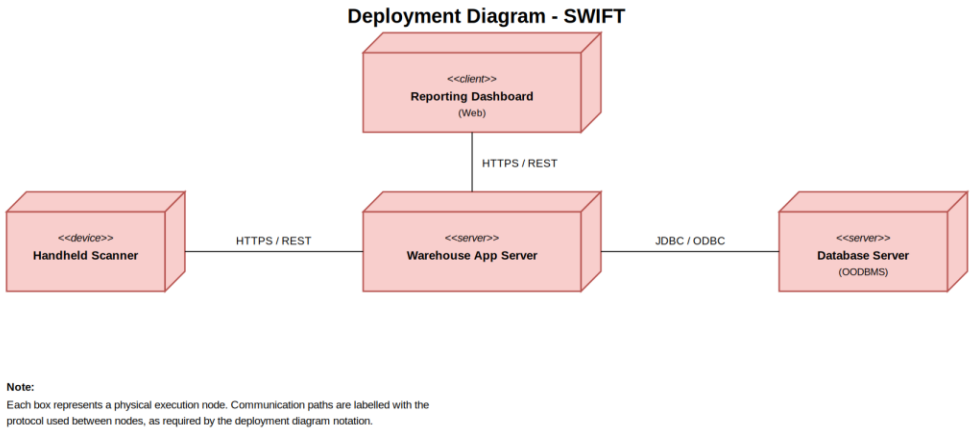

Figure 9 — Deployment Diagram

G.3.c — Design Rationale

The nine diagrams model one consistent system rather than nine unrelated pictures. The Use Case Diagram sets the scope (which actor can trigger which behaviour); the Class Diagram turns every use case into the classes and methods that implement it; the two Sequence Diagrams show two of those use cases (Place Order, Complete Picking Session) actually executing message-by-message against those same classes; the State Chart documents every value Order.status can hold, which is exactly the set of states the Activity Diagram's 'Place Order → ... → Deliver Order' flow passes through; and the Package, Component, and Deployment diagrams zoom out from the class level to show how those same responsibilities are grouped into subsystems and finally mapped onto physical hardware. Because every diagram was built from the same underlying class list, tracing a requirement from the Use Case Diagram through to the Deployment Diagram is possible end-to-end, which is exactly what the validation checklist in Section G.5 checks for.

4. G.4 — Use of Modern Tools and Techniques

All nine diagrams were prepared using draw.io (diagrams.net), a recognised UML/CASE tool, and are supplied alongside this report as editable .drawio source files so the notation, layout, and content can be independently verified and modified. Each diagram uses standard UML 2.x shapes (actor, use-case ellipse, class box with three compartments, lifeline with activation bars, state/decision nodes, package/component/node icons) rather than generic boxes and arrows.

AI-assisted tool acknowledgment: Claude (Anthropic) was used as a drafting assistant to help structure this report and generate the initial diagram layouts from the class/relationship list the student derived from the case study. The student reviewed every class, attribute, relationship, and diagram for correctness against the requirements in Section D and is able to explain and justify each design decision during evaluation, per the assignment's requirement.

Recommended practice going forward: keep the .drawio source files under version control (e.g. a Git repository alongside the report), export each diagram consistently as PNG/SVG at submission time, and maintain one shared style (colour palette, font size) across the diagram set — all of which this diagram set already follows.

5. G.5 — Results, Testing and Validation

The following checklist traces requirements across the diagram set, following the format given in the assignment brief.

Check ID	Validation Check	Diagrams Verified	Expected Result	Status / Result
VC01	Every use case maps to at least one class operation	Use Case ↔ Class	Method exists	Pass — e.g. Place Order → Order.confirmOrder(); Pick Order → PickingSession.startPicking()/completePicking()
VC02	Every class attribute referenced appears in an interaction diagram	Class ↔ Sequence	Attribute used	Pass — e.g. Order.status is read/updated in the Complete Picking Session sequence (step 8: updateStatus('Packed'))
VC03	Order state chart matches the lifecycle shown in the activity diagram	State ↔ Activity	States consistent	Pass — both show Placed → Allocated → Picking → Packed → Dispatched → Delivered plus Cancelled/Exception branches
VC04	Class-diagram multiplicities match cardinalities implied by the use cases	Class ↔ Use Case	Multiplicities match	Pass — 'Place Order' implies Customer 1—0..* Order; 'Pick Order' implies PickingSession 0..*—1 Picker
VC05	All actors in the use-case diagram appear as external entities/nodes in the deployment diagram	Use Case ↔ Deployment	Actor represented	Pass — Customer/Operator interact via the Reporting Dashboard and Handheld Scanner nodes; Picker via the Handheld Scanner
VC06	No orphan class exists without at least one relationship	Class Diagram	No isolated class	Pass — all nine classes in Figure 2 have at least one association, composition, or generalisation link

Check ID	Validation Check	Diagrams Verified	Expected Result	Status / Result
VC07	Package-diagram groupings match the class-diagram package assignment	Package ↔ Class	Grouping consistent	Pass — Order/OrderLineItem/Customer → Order Management; Item/StorageBin → Inventory Management; PickingSession/Picker → Picking & Fulfilment
VC08	Activity-diagram decision points reflect the constraints listed in Section F	Activity ↔ Constraints	Decision logic matches	Pass — 'Stock Available?' reflects the no-overselling constraint; 'Item Condition OK?' reflects the exception-handling requirement
VC09	Sequence diagram messages correspond to a real method on the target lifeline's class	Sequence ↔ Class	Method exists	Pass — every message name in Figures 3–4 (e.g. checkStock, markCompleted) matches an operation listed in Section G.3.a
VC10	Component diagram interfaces are consumed by at least one other component	Component Diagram	No dangling interface	Pass — IOrderService, IPickingService, IInventoryService, and IReportQuery are each both provided and consumed in Figure 8

6. G.6 — Analysis, Trade-offs and Engineering Justification

a. Design Justification

The class structure separates OrderLineItem from Order, PickingSession from Order, and boundary/control objects from entity objects, rather than collapsing everything into one or two large classes. Each of these separations exists because a distinct requirement in Section D1 demands it: line items need independent quantity/subtotal tracking (so they cannot be fields directly on Order), a picking session has its own lifecycle and can enter an Exception state independently of the order's own status (so it cannot simply be a status flag on Order), and separating OrderController from OrderUI means the stock-check business rule lives in exactly one place regardless of which screen or channel eventually calls it.

b. Alternative Design

Alternative considered: a single Order class holding its line items directly as an internal list of (itemId, quantity) pairs, with no separate OrderLineItem class.

- **Why the alternative is weaker:** quantity and subtotal calculations would have to live either on Order itself (bloating its responsibility) or be recomputed ad hoc wherever needed. Adding a future per-line feature — for example, a discount that applies to one line item but not the whole order — would require restructuring Order's internal list into objects anyway, so the simplification is only temporary.
- **Why OrderLineItem (chosen) is preferable:** it gives each line its own identity, its own getSubtotal() method, and a natural place to attach line-level features later, at the cost of one extra class and one extra composition relationship to document — a small price for the flexibility gained.

c. Trade-offs

- **Simplicity vs. extensibility —** the flat, mostly non-hierarchical class model (only Order is specialised, into StandardOrder/ExpressOrder) is quick to understand and implement, but a richer model — for example a common User superclass over Customer/Operator/Picker — would be more extensible if authentication and role-based permissions are added later. That hierarchy was deliberately left out of this prototype to keep the diagram set readable, at the cost of some duplicated 'who is this person' handling if roles are added in a future iteration.
- **Degree of decomposition vs. documentation overhead —** separating OrderUI/PickingUI (boundary) and OrderController/PickingController (control) from the entity classes roughly doubles the number of classes that appear across the diagram set compared to a design where a screen directly manipulates Order and Item. The separation makes each responsibility easier to test and change independently, but it does mean more classes to keep synchronised across the Class Diagram, the two Sequence Diagrams, and the Design Rationale — a documentation cost that was judged worth paying for the clearer separation of concerns.

d. Scalability Discussion

The design scales to the stated prototype target (200 items, 50 concurrent orders, 10 zones) by adding more instances of existing classes — more Item, StorageBin, and Order rows — with no change to the class structure itself, since none of the relationships are hard-coded to a specific count. For a substantially larger warehouse (thousands of items, hundreds of zones, and orders arriving continuously), the structural complexity of the class diagram itself would not need to grow, because cohesion stays high (each class still has one responsibility) and coupling stays low (entities only know about their immediate neighbours, e.g. OrderLineItem knows about Item but not about StorageBin). Two areas would need attention at that scale, however: the 'one PickingSession per Order' simplification would likely need to become 'one PickingSession per zone per Order' to let multiple pickers work a single large order in parallel, and the FulfilmentReport's aggregation queries (SLA compliance, zone utilisation) would need to move off the live transactional classes and onto a separate reporting/analytics store to avoid contention with active order processing.

e. Engineering Decision

Given the stated prototype scale, the proposed design is judged suitable as an implementation blueprint for SWIFT, provided the assumptions documented in Section G.1 (one picking session per order; report as a derived view) are carried forward explicitly into the implementation phase rather than silently assumed, and provided the two scalability caveats above are revisited if the warehouse's order volume grows well beyond the stated prototype target.

7. G.7 — Broader Considerations and Professional Responsibility

- **Sustainable and energy-efficient warehouse operation —** accurate real-time stock and bin-occupancy data (Item.currentStock, StorageBin.currentOccupancy) lets the warehouse avoid running climate-controlled storage capacity it does not need, and the zone-utilisation figures in the fulfilment report can guide consolidation of underused zones to reduce the building's overall energy footprint.
- **Reduction of overstocking and wastage —** the reorder-threshold mechanism (Item.checkReorderNeeded()) is specifically designed to flag understock before it becomes a fulfilment failure, while the same stock-level visibility helps prevent the opposite problem of overordering and wastage from expired or obsolete inventory.
- **Worker safety and fair workload distribution for pickers —** because PickingSession tracks start/completion time per picker, the system can surface workload imbalances (some pickers consistently assigned more or harder sessions than others) that a purely manual, register-based process cannot easily detect, supporting fairer shift planning and reducing fatigue-related safety risk.
- **Data privacy of customer order information —** Customer and Order data (contact and purchase details) should be accessible only to the Operator and Administrator roles that need it for fulfilment, not broadly readable across the system — this is a design responsibility to carry into the implementation phase alongside the OO structure proposed here.
- **Responsible use of AI-assisted design tools —** as acknowledged in Section G.4, an AI assistant was used to help draft this report and the diagram layouts; every design decision was reviewed against the case study's actual requirements rather than accepted uncritically, which is the standard this assignment explicitly asks students to meet.

These considerations connect to SDG 9 (Industry, Innovation and Infrastructure) through the warehouse-automation angle, and to SDG 12 (Responsible Consumption and Production) through the overstocking/wastage-reduction angle identified above.

8. G.8 — Conclusion

This report analysed the Smart Warehouse Inventory and Order Fulfilment System (SWIFT) case study and produced a complete object-oriented analysis and UML-based design: nine core classes (plus two order subtypes) identified through noun/verb analysis and classified into entity/boundary/control objects, and a full nine-diagram UML set (Use Case, Class, two Sequence, State Chart, Activity, Package, Component, and Deployment) built consistently from that same class list. The design satisfies the functional constraints in Section F — unique identifiers, no bin over-allocation, no overselling, and no double picking-session assignment — through specific encapsulated methods (Item.checkReorderNeeded(), StorageBin.allocate(), Order.confirmOrder()) rather than external validation logic. Object-oriented concepts from all three course modules were applied and justified rather than included as a notation checklist, as required.

Limitations acknowledged include the simplification of one picking session per order (rather than per zone) and treating the fulfilment report as a derived view rather than a persisted entity — both explicitly flagged in Sections G.1 and G.6 as assumptions that should be revisited if the warehouse scales substantially beyond the stated prototype target. Future improvements could introduce a User superclass over Customer/Operator/Picker to support authentication and role-based access, and a dedicated reporting/analytics data store separate from the live transactional classes.

9. G.9 — Student Reflection

> *Reflections are meant to capture your own individual learning experience — treat the drafts below as a starting point and rewrite them in your own words based on what you actually found difficult or interesting while working through this assignment.*

Reflection 1 — What did you learn beyond the concepts taught directly in class?

Beyond the UML notation itself, this assignment made clear how much of object-oriented design is really about deciding where a business rule 'lives.' The same constraint (an order cannot exceed available stock) could technically be checked in the UI, in a controller, or inside the Item class itself — working through the case study made it obvious that pushing the rule into Item.checkReorderNeeded() and Order.confirmOrder() keeps it next to the data it protects, which only becomes visible once you try to trace a requirement across multiple diagrams and notice how much simpler that tracing is when responsibilities are placed correctly the first time.

Reflection 2 — Which OOAD/UML concept was most challenging, and how was it overcome?

Keeping the Sequence Diagrams internally consistent with the Class Diagram was the hardest part — it is easy to draw a plausible-looking message exchange that does not actually correspond to any real method on the target class. This was resolved by building the class list and its operations first, then only allowing a sequence diagram message if it named a method that already existed on that list (this is exactly what validation check VC09 in Section G.5 confirms), rather than designing the diagrams independently and reconciling them afterward.

Reflection 3 — Given more time and resources, what three improvements would you make?

- Introduce a common User superclass over Customer, Operator, and Picker to support authentication and role-based permissions, which the current flat actor model does not address.
- Split PickingSession so a single large order can be picked by multiple pickers across zones in parallel, rather than assuming one session per order end-to-end.
- Move FulfilmentReport's aggregation logic onto a separate reporting data store, so heavy analytics queries do not compete with live order/picking transactions at larger scale.

Reflection 4 — How could this design extend to a network of multiple warehouses?

The current design would need a Warehouse class introduced above StorageBin, so that every bin, item stock level, and picking session is scoped to a specific warehouse rather than assumed to belong to a single global inventory. Order would then need a fulfilling-warehouse association (chosen, for example, by proximity to the customer or by which warehouse currently holds sufficient stock), and the Reporting package would aggregate across warehouses as an additional layer on top of the existing per-warehouse fulfilment report rather than replacing it.

10. G.10 — References

- Booch, G., Rumbaugh, J., & Jacobson, I. — The Unified Modeling Language User Guide (Addison-Wesley) — general UML notation reference.
- Larman, C. — Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development — reference for use-case-driven object identification and entity/boundary/control classification.
- Object Management Group (OMG) — UML 2.x Specification, https://www.omg.org/spec/UML/ — authoritative UML notation documentation.
- draw.io / diagrams.net — https://app.diagrams.net — UML/CASE tool used to prepare all nine diagrams in this report.
- Course materials — CSA11 Object Oriented Analysis and Design, Modules I–III (OOSDLC, UML diagram types, OO analysis techniques).
- AI-assisted tool: Claude (Anthropic) was used to help draft this report's structure and the initial UML diagram layouts, as acknowledged in Section G.4; all content was reviewed by the student against the case study requirements.